

## 0.108. Une lettre qui tire — des étincelles

La rafale du 0.105.1 : chaque tir est un point, et il casse un seul X, celui qu'il touche en face. Au choc, un petit éclat jaillit : six étincelles de  $2 \times 2$  pixels, jaunes et orange, qui scintillent un instant puis disparaissent.

### 1. Ce qu'il faut comprendre

**On reprend la rafale du 0.105.1**, et on lui ajoute un effet. Le tir est un **point** « . » ; il casse **un seul X**, celui qu'il touche **en face**, et disparaît. Au moment du choc, un petit **éclat** jaillit du trou : six étincelles jaunes et orange, qui scintillent un instant, puis s'éteignent.

**Ce qui est nouveau ici (1) : une étincelle est un lutin.** Une lettre ne bouge que case par case : 8 pixels d'un coup. Un **lutin** (le 0.70) se pose **au pixel près**, avec `sprite(numero, x, y, dessin)`. Les six étincelles sont les lutins 0 à 5 ; leur place se compte en **pixels** : `px[i]`, `py[i]`. La case (c, l) commence au pixel ( $c \times 8, l \times 8$ ).

**Ce qui est nouveau ici (2) : un dessin à nous, Tuile ETINCELLE.** Huit lignes de huit chiffres : un chiffre par pixel. 0 est **transparent** (on voit le décor derrière) ; 1 et 2 sont deux couleurs. Seuls quatre pixels, au milieu, sont allumés :  $2 \times 2$ , une étincelle.

**Ce qui est nouveau ici (3) : les couleurs.** `couleurLutin(paLETTE, teinte, rouge, vert, bleu)` règle une teinte d'une palette de lutins, avec trois nombres de 0 à 31. La palette 0 met le jaune (31, 31, 6) en 1 et l'orange (31, 16, 0) en 2 ; la palette 1 les échange. `teindreLutin(i, palette)` choisit la palette du lutin i. Le programme s'en sert : la console passe **toute seule** en mode couleur.

**Ce qui est nouveau ici (4) : `ecLat(c, 1)`.** Quand un tir casse le X en (`nx`, `ny`), on appelle `ecLat(nx, ny)` : les six étincelles sont posées au centre de cette case, et chacune reçoit une **vie** de 6 à 11 images (`6 + hasard() % 6`). Elles ne s'éteignent pas toutes ensemble : c'est ce qui fait « étincelles ».

**`etinceler()`, à chaque image** : chaque étincelle encore vivante perd une image de vie, avance d'un pixel dans sa direction (`PDX[i]`, `PDY[i]`, gravées dans la cartouche : 255 veut dire -1, le 0.100), est redessinée, et **scintille** : `teindreLutin(i, ((images() + i) / 2) % 2)` la fait passer du jaune à l'orange toutes les 2 images. À 0, `cacher(i)` la fait disparaître.

**Un seul X, celui d'en face** : rien ne change dans le pas du tir du 0.105.1. Son arrivée est un X : `effacer(nx, ny, 1)` efface **cette case-là**, le tir disparaît, et l'éclat part de là. Les X au-dessus et au-dessous ne sont pas touchés.

**Déroulons** : A en (2, 8), bouton A. Le point va de (3, 8) à (11, 8). Pas suivant : l'arrivée (12, 8) est un X. Il est effacé, le tir disparaît, et `ecLat(12, 8)` pose les six étincelles au pixel (96, 64). Une image plus tard, elles s'écartent d'un pixel : deux en haut, deux en bas, deux de chaque côté. Dix images plus tard, il n'en reste plus.

**Essaie** : change les couleurs (du bleu et du blanc pour de la glace : 10, 20, 31 et 31, 31, 31). Ou un éclat plus long : `12 + hasard() % 8`.

## Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <hasard>` → [0.81.2. La fonction hasard\(\) — un nombre au hasard](#)
- `#include <caler>` → [0.107.3. La fonction caler\(\) — ôter un lutin](#)
- `#include <sprite>` → [0.69.4. La fonction sprite\(\) — un lutin au pixel près](#)
- `#include <teindreLutin>` → [0.107.5. La fonction teindreLutin\(\) — un lutin dans une palette](#)
- `#include <couleurLutin>` → [0.107.4. La fonction couleurLutin\(\) — une couleur des personnages](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <chaque>` → [0.29.3. La fonction chaque\(\) — toutes les n millisecondes](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <lire>` → [0.68.4. La fonction lire\(\) — lire une case](#)
- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <reste>` → [0.8.4. #include <reste> — a % b](#)

## 2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1655 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

|                    |       |                                                                                |
|--------------------|-------|--------------------------------------------------------------------------------|
| démarrage          | 90 o  | ← la console, à l'allumage - la mise en route de la console                    |
| chaque_minuteur()  | 124 o | ← main() - une fonction fournie par le langage                                 |
| eclat()            | 78 o  | ← main() - ta fonction                                                         |
| etinceler()        | 247 o | ← main() - ta fonction                                                         |
| main()             | 762 o | ← démarrage - ta fonction                                                      |
| VBlank             | 8 o   | ← la console, à chaque image - compte les images, 60 fois par seconde          |
| AttendreImage      | 28 o  | ← image() - attend l'image suivante                                            |
| AttendreVBlank     | 16 o  | ← démarrage - attend que l'écran soit libre                                    |
| AttendreAcces      | 12 o  | ← EffacerCases, EcrireTexte - attend de pouvoir écrire une case                |
| AdresseCase        | 10 o  | ← poser(), lire(), effacer(), texte() - trouve l'adresse d'une case de l'écran |
| EffacerCases       | 12 o  | ← effacer() - efface des cases                                                 |
| EcrireTexte        | 10 o  | ← texte() - écrit un texte, lettre par lettre                                  |
| Reste              | 10 o  | ← eclat() - le reste d'une division                                            |
| RangerLutins       | 11 o  | ← démarrage - range les 40 lutins hors de l'écran                              |
| InstallerTransfert | 15 o  | ← démarrage - installe la copie des lutins en mémoire rapide                   |
| CopierTuiles       | 58 o  | ← démarrage - copie les lettres et les dessins en mémoire vidéo                |
| EffacerCarte       | 14 o  | ← démarrage - vide les deux cartes de l'écran                                  |
| Hasard             | 25 o  | ← hasard() - tire un nombre au hasard                                          |
| LireManette        | 37 o  | ← bouton() - lit les huit boutons                                              |
| Tuiles             | 64 o  | ← CopierTuiles - 3 lettres sur 43, et 1 tuile dessinée                         |
| DonneesTransfert   | 10 o  | ← InstallerTransfert - la petite routine qui copie les lutins                  |
| "."                | 1 o   | ← texte() - un texte gravé                                                     |
| "."                | 1 o   | ← texte() - un texte gravé                                                     |
| PDX_0              | 6 o   | ← etinceler() - une table gravée                                               |
| PDY_1              | 6 o   | ← etinceler() - une table gravée                                               |

Lettres gravées : AX.

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireNombre, Diviser, Multiplier, DecalerGauche, DecalerDroite, EffacerFond, EffacerPanneau, Notes

Fonctions du langage appelées :

|          |     |                                                                      |
|----------|-----|----------------------------------------------------------------------|
| texte()  | × 2 | 68 o sur place, dans main() + AdresseCase (10 o), EcrireTexte (10 o) |
| sprite() | × 1 | 62 o sur place, dans etinceler()                                     |

|                |     |                                                                       |
|----------------|-----|-----------------------------------------------------------------------|
| effacer()      | x 3 | 61 o sur place, dans main() + AdresseCase (10 o), EffacerCases (12 o) |
| couleurLutin() | x 4 | 48 o sur place, dans main()                                           |
| lire()         | x 4 | 44 o sur place, dans main() + AdresseCase (10 o)                      |
| poser()        | x 3 | 38 o sur place, dans main() + AdresseCase (10 o)                      |
| teindreLutin() | x 1 | 33 o sur place, dans etinceler()                                      |
| bouton()       | x 5 | 25 o sur place, dans main() + LireManette (37 o)                      |
| cache()        | x 1 | 13 o sur place, dans etinceler()                                      |
| image()        | x 1 | 6 o sur place, dans main() + AttendreImage (28 o)                     |
| images()       | x 2 | 4 o sur place, dans chaque_minuteur(), etinceler()                    |
| hasard()       | x 1 | 3 o sur place, dans eclat() + Hasard (25 o)                           |

Tes fonctions appelées :

|                   |     |                             |
|-------------------|-----|-----------------------------|
| chaque_minuteur() | x 3 | 32 o sur place, dans main() |
| eclat()           | x 1 | 11 o sur place, dans main() |
| etinceler()       | x 1 | 3 o sur place, dans main()  |

## 1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- UNE LETTRE QUI TIRE : des étincelles ----
// La rafale du 0.105.1 : chaque tir est un point « . », et il casse UN X,
// celui qu'il touche en face. NOUVEAU : au choc, un petit ÉCLAT jaillit de
// l'endroit touché – six étincelles de 2 x 2 pixels, jaunes et orange, qui
// scintillent un instant, puis disparaissent.
// Une étincelle est un LUTIN (le 0.70) : un petit dessin placé au pixel près.
// ALPHABET[23] : le X.

#include <Tuile>           // un dessin de 8 x 8 pixels
#include <hasard>          // tire un nombre au hasard
#include <cache>           // cache un lutin
#include <sprite>          // place un lutin de 8 x 8 au pixel près
#include <teindreLutin>    // met un lutin dans une palette
#include <couleurLutin>    // choisit une couleur d'une palette des lutins
#include <poser>           // pose une tuile sur une case du fond
#include <ALPHABET>        // les lettres de la police : ALPHABET[0] est le A
#include <chaque>          // répond 1 toutes les n millisecondes
#include <bouton>          // lit un bouton de la manette
#include <lire>            // lit la tuile posée sur une case
#include <effacer>         // efface des cases, ou tout le fond
#include <texte>           // écrit un texte à l'écran
#include <reste>           // a % b, sauf par 1, 2, 4, 8, 16... écrits en clair
```

## 2. UNE VARIABLE — AX

```
uint8_t ax = 2;           // le A : sa colonne...
```

## 3. UNE VARIABLE — AY

```
uint8_t ay = 8;           // ...et sa ligne
```

## 4. UNE VARIABLE — DX

```
uint8_t dx = 1;           // le sens où regarde le A
```

## 5. UNE VARIABLE — DY

```
uint8_t dy = 0;
```

## 6. UNE VARIABLE — NX

```
uint8_t nx = 0;           // une case d'ARRIVÉE, calculée avant de bouger
```

## 7. UNE VARIABLE — NY

```
uint8_t ny = 0;
```

## 8. UN TABLEAU — TX

```
uint8_t tx[3];           // les trois tirs : leurs colonnes...
```

## 9. UN TABLEAU — TY

```
uint8_t ty[3];           // ...leurs lignes...
```

## 10. UN TABLEAU — TDX

```
uint8_t tdx[3];          // ...et leurs sens
```

## 11. UN TABLEAU — TDY

```
uint8_t tdy[3];
```

## 12. UN TABLEAU — VOLE

```
uint8_t vole[3];         // vole[i] = 1 : le tir n° i est en vol
```

## 13. UNE VARIABLE — LANCE

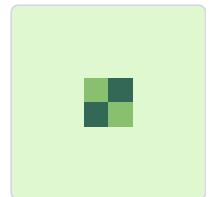
```
uint8_t lance = 0;

// ---- NOUVEAU : le dessin d'une étincelle, 2 × 2 pixels au milieu ----
// 0 : transparent (on voit le décor derrière). 1 et 2 : deux couleurs,
// réglées plus bas avec couleurLutin().
```

## 14. UN DESSIN — ETINCELLE

```
Tuile ETINCELLE = {
    "00000000",
    "00000000",
    "00000000",
    "00012000",
    "00021000",
    "00000000",
    "00000000",
    "00000000",
};

// ---- NOUVEAU : six étincelles, rangées dans des tableaux ----
```



à l'écran

## 15. UN TABLEAU — PX

```
uint8_t px[6];           // leur place, en PIXELS (et non en cases)
```

## 16. UN TABLEAU — PY

```
uint8_t py[6];
```

## 17. UN TABLEAU — VIE

```
uint8_t vie[6];           // les images qu'il leur reste à briller (0 : éteinte)

// ---- NOUVEAU : leurs six directions, gravées dans la cartouche ----
// De combien de pixels chacune avance à chaque image. 255, c'est -1
// (un uint8_t fait le tour, le 0.100) :
// 0 : à droite, un peu en haut    3 : à gauche, un peu en bas
// 1 : à droite, un peu en bas    4 : tout droit en haut
// 2 : à gauche, un peu en haut   5 : tout droit en bas
```

## 18. UNE TABLE GRAVÉE DANS LA CARTOUCHE — PDX

```
const uint8_t PDX[] = { 1, 1, 255, 255, 0, 0 };
```

## 19. UNE TABLE GRAVÉE DANS LA CARTOUCHE — PDY

```
const uint8_t PDY[] = { 255, 1, 255, 1, 255, 1 };

// ---- NOUVEAU : un éclat, au centre de la case (c, l) ----
// Une case fait 8 pixels : la case (c, l) commence au pixel (c × 8, l × 8).
```

## 20. UNE FONCTION — ECLAT()

```
void eclat(uint8_t c, uint8_t l) {
    for (uint8_t i = 0; i < 6; i++) {
        px[i] = c * 8;
        py[i] = l * 8;
        vie[i] = 6 + hasard() % 6;      // de 6 à 11 images : un instant
    }
}

// ---- NOUVEAU : faire vivre les étincelles, une image de plus ----
```

## 21. UNE FONCTION — ETINCELER()

```
void etinceler() {
    for (uint8_t i = 0; i < 6; i++) {
        if (vie[i] > 0) {
            vie[i] = vie[i] - 1;
            if (vie[i] == 0) {
                cacher(i);                // plus de vie : le lutin i disparaît
            } else {
                px[i] = px[i] + PDX[i];    // elle file dans SA direction...
                py[i] = py[i] + PDY[i];
                sprite(i, px[i], py[i], ETINCELLE);
                // ...et scintille : toutes les 2 images, elle passe d'une palette à
                // l'autre (0 : jaune au milieu, 1 : orange). Après sprite(), qui
                // remet la palette à 0.
                teindreLutin(i, ((images() + i) / 2) % 2);
            }
        }
    }
}
```

## 22. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // ---- NOUVEAU : les deux palettes des étincelles (couleurs de 0 à 31) ----
    couleurLutin(0, 1, 31, 31, 6);    // palette 0 : jaune...
    couleurLutin(0, 2, 31, 16, 0);    // ...et orange
    couleurLutin(1, 1, 31, 16, 0);    // palette 1 : orange...
    couleurLutin(1, 2, 31, 31, 6);    // ...et jaune : les mêmes, échangées

    for (uint8_t l = 4; l < 13; l++) {    // le mur : 9 X en colonne 12 (le 0.101)
        poser(12, l, ALPHABET[23]);
    }
    poser(ax, ay, ALPHABET[0]);

    while (true) {
        image();

        // ---- le A bouge et retient où il regarde ----
        if (chaque(150)) {
            nx = ax;
            ny = ay;
            if (bouton(DROITE)) {
                nx = ax + 1;
                dx = 1;
                dy = 0;
            } else if (bouton(GAUCHE)) {
                nx = ax - 1;
                dx = -1;
                dy = 0;
            } else if (bouton(BAS)) {
                ny = ay + 1;
                dx = 0;
                dy = 1;
            } else if (bouton(HAUT)) {
                ny = ay - 1;
                dx = 0;
                dy = -1;
            }
            if (nx < 20 && ny < 17) {
                if (lire(nx, ny) == 0) {
                    effacer(ax, ay, 1);
                    ax = nx;
                    ay = ny;
                    poser(ax, ay, ALPHABET[0]);
                }
            }
        }

        // ---- le bouton A lance un tir, au plus toutes les 250 ms ----
        if (chaque(250)) {
            if (bouton(A)) {
                nx = ax + dx;
                ny = ay + dy;
                // la case de départ doit être dans l'écran ET vide (le 0.101)
                if (nx < 20 && ny < 17 && lire(nx, ny) == 0) {
                    lance = 0;
                    for (uint8_t i = 0; i < 3; i++) {
                        if (lance == 0 && vole[i] == 0) {
                            vole[i] = 1;
                            tx[i] = nx;
                            ty[i] = ny;
                            tdx[i] = dx;
                            tdy[i] = dy;
                            texte(tx[i], ty[i], ".");
                            lance = 1;
                        }
                    }
                }
            }
        }
    }
}
```

```

    }
  }
}

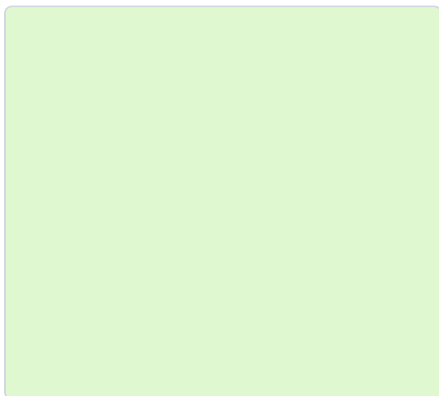
// ---- chaque tir avance, et regarde sa case d'arrivée (le 0.105) ----
if (chaque(100)) {
  for (uint8_t i = 0; i < 3; i++) {
    if (vole[i] == 1) {
      nx = tx[i] + tdx[i];
      ny = ty[i] + tdy[i];
      effacer(tx[i], ty[i], 1);
      if (nx >= 20 || ny >= 17) {
        vole[i] = 0; // 1. hors de l'écran
      } else if (lire(nx, ny) == ALPHABET[23]) {
        effacer(nx, ny, 1); // 2. un X : le tir casse CE X-là, et lui seul...
        vole[i] = 0; // ...disparaît...
        eclat(nx, ny); // ...et NOUVEAU : un éclat jaillit du choc
      } else if (lire(nx, ny) != 0) {
        vole[i] = 0; // 3. un autre tir : il s'écrase
      } else {
        tx[i] = nx; // 4. une case vide : il y va
        ty[i] = ny;
        texte(tx[i], ty[i], ".");
      }
    }
  }
}

// ---- NOUVEAU : les étincelles vivent, une image après l'autre ----
etinceler();
}
}

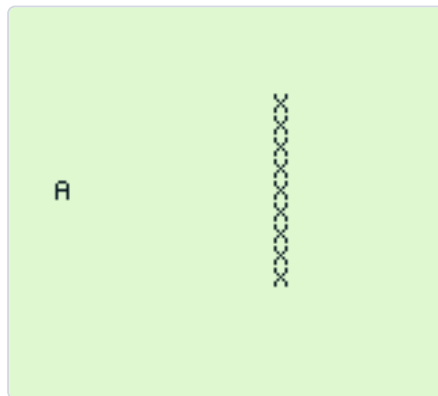
```

### 3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



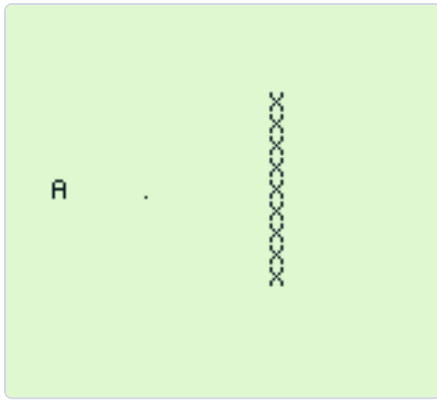
la toute première image — la console vient de démarrer — image 1



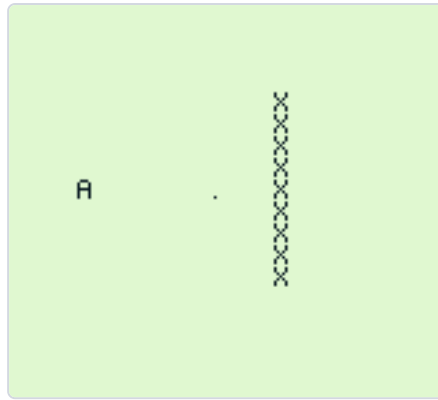
un tiers de seconde plus tard : le premier dessin est posé — image 20

### 4. Ce que les touches changent

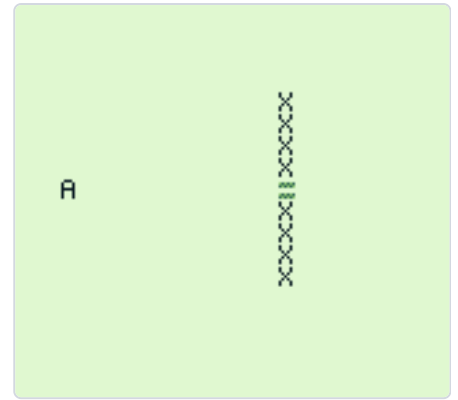
Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



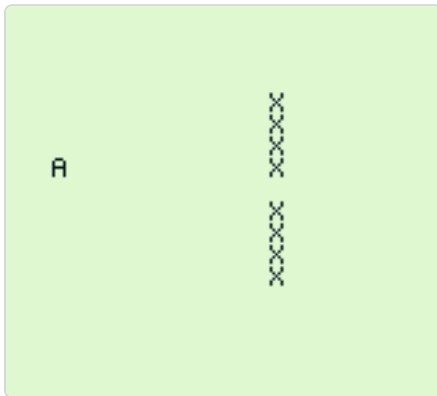
**A** — l'écran a changé



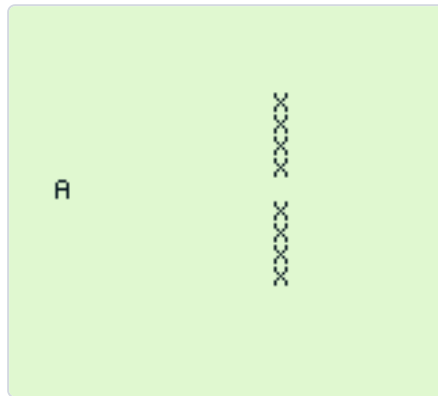
**DROITE** — l'écran a changé



**GAUCHE** — l'écran a changé



**HAUT** — l'écran a changé



**BAS** — l'écran a changé

## 5. Ce qu'on doit voir

Chaque point de la rafale casse le X qu'il touche en face, et lui seul ; au choc, un petit éclat jaune et orange jaillit du trou, scintille un instant, puis disparaît.

## 6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ avant le choc : ni jaune ni orange
- ✓ le tir casse un seul X, celui d'en face (12, 8) ; les autres restent
- ✓ au choc, un éclat jaune et orange, tout près du trou
- ✓ un instant plus tard, il a disparu

Ensuite : **0.109. La fonction `changerDessin()` — redessiner une tuile partout** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++